

UNIDAD 2 - LA VPS: APROVISIONAR, ENTRAR Y CERRAR

Taller de aula: cerrar un servidor sin tener el servidor

Programacion 3 - Tecnicatura Universitaria en Programacion - UTN FRM - CLAVE DE CORRECCION DOCENTE

Modalidad	En grupos de dos o tres, en el aula. Se entrega antes de irse : lo que no se termina en la mesa no se termina.
Requisitos	No hace falta tener el VPS provisionado. Tampoco hace falta internet. Solo el apunte del capitulo 2 a mano.
Tiempo	70 minutos. Partes 1 a 5, mas la Parte 6 que se lleva al proyecto.
Puntaje	100 puntos: 1 (25) + 2 (15) + 3 (20) + 4 (15) + 5 (15) + 6 (10).
Que se evalua	Criterio operativo: decidir que se expone y que no, y poder justificar cada decision con el mecanismo que la explica.
Integrantes	

Antes de arrancar

Este taller no pregunta que recordas: pregunta que **decidirias**. Todas las salidas de comandos que vas a leer estan impresas aca; no hay nada que ejecutar. Cuando dudes, busca la seccion del apunte: encontrarla tambien es parte del ejercicio. Y una regla de oro para todo el taller: **una decision sin justificacion no suma puntaje**. "Porque si" no es una razon de ingenieria.

Parte 1 - Inventario de puertos y decision de exposicion

25 puntos

Un grupo de la comision levanto su proyecto en el VPS y corrio `ss -tlnp`. Esta es la salida completa, sin recortar. El proyecto tiene una API en FastAPI, una base MongoDB, un Redis de cache y el panel de Easypanel.

```
root@vps-tup:~# ss -tlnp
State  Recv-Q  Send-Q  Local Address:Port  Process
LISTEN  0        4096    0.0.0.0:22      users:((("sshd",pid=791))
LISTEN  0        4096    0.0.0.0:80      users:((("traefik",pid=1620))
LISTEN  0        4096    0.0.0.0:443     users:((("traefik",pid=1620))
LISTEN  0        4096    0.0.0.0:3000    users:((("docker-proxy",pid=1704))
LISTEN  0        4096    0.0.0.0:8000    users:((("docker-proxy",pid=1893))
LISTEN  0        4096    0.0.0.0:27017   users:((("docker-proxy",pid=1955))
LISTEN  0        511     127.0.0.1:6379   users:((("redis-server",pid=1512))
```

1.1 Completa el inventario

14 puntos (2 por fila)

Una fila por puerto. En "Rango RFC 6335" escribi si es bien conocido, registrado o dinamico. En "Alcanzable desde internet" respondes lo que **deberia** ser, no lo que hoy es.

CLAVE DE CORRECCION

La columna decisiva es la de interfaz: `0.0.0.0` significa "todas las interfaces, incluida la publica"; `127.0.0.1` es solo loopback (seccion 2.6.1). Ver tabla completa mas abajo. Se otorga el punto completo solo si la justificacion nombra el mecanismo, no si repite el sintoma.

Puerto	Proceso	Interfaz en la que escucha	Rango RFC 6335	Debe ser alcanzable desde internet?	Por que (una linea)
22	sshd	Todas (0.0.0.0)	Bien conocido	Si	Es la unica puerta de administracion. Se deja abierto, pero con clave unicamente y fail2ban detras (2.11).
80	traefik	Todas (0.0.0.0)	Bien conocido	Si	Redirige a HTTPS y, sobre todo, por el 80 pasa el desafio de Let's Encrypt : si lo cerras, no se emite el certificado (2.10 y tabla de errores frecuentes).
443	traefik	Todas (0.0.0.0)	Bien conocido	Si	Es por donde entra todo el trafico legitimo de la aplicacion.
3000	docker-proxy (panel)	Todas (0.0.0.0)	Registrado	No	El panel se publica por subdominio con HTTPS, no por IP: 3000. Publicado asi viaja sin cifrar y queda indexado en horas (2.9.2, 2.10).
8000	docker-proxy (API)	Todas (0.0.0.0)	Registrado	No	El mundo llega a la API por el proxy inverso en el 443 . Un puerto propio publicado es una segunda puerta sin TLS ni reglas (2.8.3).
27017	docker-proxy (MongoDB)	Todas (0.0.0.0)	Registrado	No, nunca	Base de datos publicada a internet: el caso textual de 2.9.2. Se descubre por escaneo masivo el mismo dia.
6379	redis-server	Solo loopback (127.0.0.1)	Registrado	No, y ya esta bien	Escucha solo en loopback: es inalcanzable desde internet aunque el firewall este apagado (2.6.1). Es el unico de la lista que no hay que tocar.

1.2 Elegi la estrategia de cierre

6 puntos (2 por servicio)

Para los tres servicios que decidiste que **no** deben verse desde internet pero hoy escuchan en 0.0.0.0, elegi una de las estrategias de la seccion 2.8.3 y justificala. Ojo: "agregar una regla ufw deny" no es una de ellas, y en la Parte 2 vas a ver por que.

CLAVE DE CORRECCION

3000 (panel): dejar de publicarlo y exponerlo por subdominio a traves del proxy inverso, con certificado (easypanel.tudominio.me). **8000 (API)**: no publicar el puerto; el contenedor se une a la red interna del proyecto y Traefik lo alcanza por nombre de servicio. **27017 (Mongo)**: no publicar el puerto nunca; solo red interna, alcanzable por la API y por nadie mas. La estrategia general es la misma en los tres casos y es la conclusion del capitulo: **el puerto mas seguro es el que nunca se publica**. Se acepta tambien, para acceso administrativo puntual a la base, el tunel SSH con -L (2.8.3, segunda estrategia).

1.3 Predecí la evidencia externa

5 puntos

Suponiendo que el grupo aplica exactamente las correcciones que propusiste, escribi la salida que debería dar `nmap -Pn tudominio.me` corrido desde otra red. Una línea por puerto. Y decime por que esta comprobacion vale mas que cualquier otra del capitulo.

CLAVE DE CORRECCION

Debe listar exactamente **tres** puertos abiertos: 22/tcp open ssh, 80/tcp open http, 443/tcp open https. Nada mas. Vale mas que las demas porque es la **única evidencia externa** (2.13): las otras seis comprobaciones describen intenciones declaradas dentro del servidor; `nmap` desde afuera describe hechos. La bandera `-Pn` evita que `nmap` intente primero un ping de descubrimiento, que muchos proveedores descartan (2.8.2).

Parte 2 - El recorrido del paquete

15 puntos

Dos paquetes salen de la misma notebook, en la misma red, con un segundo de diferencia. Los dos llegan a la IP publica del mismo servidor. Uno lo filtra `ufw` y el otro no. No es magia ni un error de Ubuntu: es el recorrido.

```
Paquete A  destino 203.0.113.45:22      -> sshd, que corre en el host
Paquete B  destino 203.0.113.45:27017   -> mongo, que corre en un contenedor
                                                publicado con -p 27017:27017
```

2.1 Traza los dos recorridos

8 puntos (4 por paquete)

Complete la secuencia de puntos de enganche de netfilter que atraviesa cada paquete, desde que entra por la placa de red hasta que llega al proceso. Marca con un circulo el momento exacto en que los dos caminos se separan.

CLAVE DE CORRECCION

Paquete A: placa de red » PREROUTING » decision de enrutamiento: "el destino soy yo" » **INPUT** » sshd. **Paquete B:** placa de red » PREROUTING, donde la regla DNAT que escribio Docker reescribe el destino a la IP del contenedor (por ejemplo 172.18.0.4:27017) » decision de enrutamiento: "el destino **ya no soy yo**" » **FORWARD** » POSTROUTING » contenedor. Los caminos se separan en la **decision de enrutamiento**, y se separan porque el DNAT previo cambio el destino (2.7.1 y 2.8.1).

	Paquete A (puerto 22, proceso en el host)	Paquete B (puerto 27017, contenedor publicado)
1	Llega por la placa de red	Llega por la placa de red
2	PREROUTING (sin cambios de destino)	PREROUTING: DNAT de Docker reescribe el destino a la IP del contenedor
3	Decision de enrutamiento: es para esta maquina	Decision de enrutamiento: ya no es para esta maquina
4	INPUT (aca escribe sus reglas ufw)	FORWARD (aca ufw no escribio nada) » POSTROUTING
5	Lo recibe sshd	Lo recibe el proceso dentro del contenedor

2.2 Explicalo en una frase de ingenieria

4 puntos

Completa: "sudo ufw deny 27017 no bloquea nada porque ufw escribe sus reglas en la cadena _____, y el paquete publicado por Docker pasa por la cadena _____, de modo que _____."

CLAVE DE CORRECCION

INPUT / FORWARD / la regla nunca llega a evaluarse contra ese paquete. Descontar si el alumno escribe "porque Docker desactiva ufw" o "porque es un bug": no lo desactiva y no es un bug, es la consecuencia previsible del recorrido (2.8.1).

2.3 Que comando muestra la verdad de adentro

3 puntos

Nombra el comando que permite leer el conjunto de reglas real del nucleo, con las cadenas que creo ufw y las que creo Docker una al lado de la otra. Y explica por que ese comando, aun siendo mas honesto que ufw status, **sigue sin ser** la verificacion definitiva.

CLAVE DE CORRECCION

sudo nft list ruleset (2.12.2 y actividad 6). Sigue sin ser definitivo porque tambien se ejecuta **dentro** del servidor: muestra que reglas hay, no que ve el mundo. Podes tener el conjunto de reglas perfecto y un proveedor, un grupo de seguridad de nube o un NAT intermedio cambiando el resultado. La unica prueba de lo que ve el mundo se toma desde el mundo (2.13).

Parte 3 - El guion de endurecimiento

20 puntos

Doce pasos, desordenados, sobre un servidor recién creado al que hoy se entra con contraseña de root. El objetivo es dejarlo con acceso solo por clave y con el firewall activo, **sin quedarse afuera en el intento**. Este ejercicio es el que mas alumnos pierde en la practica: el orden no es una preferencia de estilo, es la diferencia entre tener servidor y no tenerlo.

- a) sudo ufw allow OpenSSH
- b) sudo ufw enable
- c) ssh-keygen -t ed25519 -C "matias@notebook"
- d) editar /etc/ssh/sshd_config y poner PasswordAuthentication no
- e) sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
- f) ssh-copy-id root@203.0.113.45
- g) sudo systemctl restart ssh
- h) abrir una SEGUNDA terminal y entrar por SSH, sin cerrar la primera
- i) sudo apt update && sudo apt install fail2ban -y
- j) sudo ufw status verbose
- k) nmap -Pn tudominio.me (desde otra maquina, en otra red)
- l) ssh root@203.0.113.45 y comprobar que NO pide contraseña

3.1 Ordena los doce pasos

10 puntos

Escribi las doce letras en el orden correcto de ejecucion.

CLAVE DE CORRECCION

c - f - l - a - e - b - j - d - g - h - i - k. Se genera el par (c), se instala la clave publica (f), se verifica que la clave funciona **antes** de tocar nada (l), se permite SSH en ufw (a), se permiten 80 y 443 (e), recién entonces se habilita el firewall (b), se controla lo declarado (j), se deshabilita la contraseña (d), se reinicia el servicio (g), se verifica desde una segunda sesion sin cerrar la actual (h), se agrega fail2ban (i) y se cierra con la evidencia externa (k). Se acepta e/a intercambiados y j en otro punto posterior a b; **no** se acepta ningun orden en el que b vaya antes que a, ni g antes que l.

3.2 Marca los tres puntos de no retorno

6 puntos (2 por punto)

Tres de esos pasos, ejecutados fuera de orden, te dejan afuera del servidor y solo se arreglan por la consola de emergencia de Hostinger. Identificalos y decime en cada caso que error concreto los convierte en trampa.

CLAVE DE CORRECCION

(b) ufw enable sin haber corrido antes (a): el perfil predeterminado es deny (incoming), la sesion SSH en curso se congela y no hay forma de volver a entrar. **(g) reiniciar SSH despues de (d) sin haber verificado (l):** si la clave publica no quedo bien instalada, acabas de eliminar el unico metodo de acceso que funcionaba. **(f) mal ejecutado:** la clave se copia al usuario equivocado o al authorized_keys equivocado, y el error **no se manifiesta ahi**, se manifiesta recién en (g), cuando ya es tarde. Los tres estan en la tabla de errores frecuentes de 2.14.

3.3 La red de seguridad

4 puntos

Dos de los doce pasos no cambian absolutamente nada en el servidor: solo miran. Identificalos y explica que principio de trabajo representan y por que van exactamente donde los pusiste.

CLAVE DE CORRECCION

(l) y (h) -en sentido amplio tambien (j) y (k)- no modifican estado: verifican. Representan la regla que atraviesa toda la clase: **verificar antes de cerrar la puerta, y verificar desde una sesion distinta de la que estas usando.** (h) es el clasico "no cierres la terminal con la que entraste hasta comprobar que podes entrar de nuevo": mientras esa sesion siga abierta, cualquier error es reversible desde adentro.

Parte 4 - Auditoria de un archivo de configuracion

15 puntos

Un grupo entrego este fragmento de /etc/ssh/sshd_config como "servidor endurecido". Tiene **cinco problemas reales** y **una linea que parece un problema y no lo es.**

```
# /etc/ssh/sshd_config (fragmento entregado por un grupo)
1  Port 22
2  PermitRootLogin yes
3  PubkeyAuthentication yes
4  PasswordAuthentication yes
5  PermitEmptyPasswords yes
6  MaxAuthTries 30
7  X11Forwarding yes
8  AuthorizedKeysFile .ssh/authorized_keys
9  UsePAM yes
```

4.1 Los cinco problemas

10 puntos (2 por problema)

Para cada uno: numero de linea, valor corregido y una linea de justificacion. Sin justificacion no suma.

CLAVE DE CORRECCION

Linea 2 PermitRootLogin yes » prohibit-password: permite entrar como root **con contrasena**, que es justamente lo que se quiere eliminar. Con prohibit-password root sigue entrando, pero solo por clave (2.5.5). **Linea 4** PasswordAuthentication yes » no: mientras siga en yes, toda la fuerza bruta de 2.9.1 sigue teniendo una puerta contra la cual probar. **Linea 5** PermitEmptyPasswords yes » no: es el problema mas grave del archivo. Habilita cuentas con contrasena vacia. **Linea 6** MaxAuthTries 30 » 3 a 6: treinta intentos por conexion multiplica por diez el trabajo util de cada intento de fuerza bruta. **Linea 7** X11Forwarding yes » no: un servidor sin entorno grafico no necesita reenvio de X11. Es superficie de ataque gratuita: **economia del mecanismo** (2.12.1). La linea 3 (PubkeyAuthentication yes) es correcta y debe quedar; si un alumno la marca como error, no entendio el mecanismo.

4.2 La linea que no es un problema

5 puntos

Un integrante propone cambiar la linea 1 a Port 2222 "para que no lo encuentren los bots". Identifica por que esa medida **no** mejora la seguridad del servidor en terminos del capitulo, y nombra ademas el riesgo operativo concreto que introduce si se aplica en el orden equivocado.

CLAVE DE CORRECCION

Es **seguridad por oscuridad**: reduce el ruido en los registros, no el riesgo. El escaneo de 2.9.1 no busca el puerto 22, busca **todos** los puertos; un escaneo completo encuentra el 2222 en segundos. Lo que hace fuerte al servidor es que la contrasena no sea un metodo valido, no donde este la puerta. Riesgo operativo: si cambias el puerto y reinicias SSH **sin haber corrido antes** sudo ufw allow 2222/tcp, el firewall sigue permitiendo solo el 22 y te quedas afuera. Es el mismo punto de no retorno de la Parte 3, con otra ropa. Se acepta como matiz valido que reduce volumen de log y por lo tanto hace mas legible una intrusion real, siempre que el alumno diga explicitamente que **no es una medida de seguridad**.

Parte 5 - Peritaje: reconstruir un incidente

15 puntos

Un grupo pide ayuda: "nos borraron una tabla de la base y no entendemos como, si el SSH estaba blindado". Esta es toda la evidencia que trajeron. Leela entera antes de escribir nada.

```
root@vps-tup:~# docker ps --format "{{.Names}}\t{{.Ports}}"
tup-api      0.0.0.0:8000->8000/tcp
tup-db       0.0.0.0:5432->5432/tcp
tup-front    0.0.0.0:8080->80/tcp

root@vps-tup:~# sudo ufw status
Estado: activo
Hasta      Accion      Desde
22/tcp     ALLOW IN    Anywhere
80/tcp     ALLOW IN    Anywhere
443/tcp    ALLOW IN    Anywhere
5432/tcp   DENY IN     Anywhere

--- desde la notebook de un companero, en otra red ---
$ nmap -Pn 203.0.113.45
PORT      STATE      SERVICE
22/tcp    open      ssh
80/tcp    open      http
443/tcp   open      https
5432/tcp  open      postgresql
8000/tcp  open      http-alt
8080/tcp  open      http-proxy

root@vps-tup:~# grep -c "Failed password" /var/log/auth.log
0

--- registro del contenedor de la base ---
2026-08-14 03:12:07 UTC [4412] FATAL: password authentication failed for user "postgres"
2026-08-14 03:12:09 UTC [4415] FATAL: password authentication failed for user "postgres"
2026-08-14 03:12:11 UTC [4419] FATAL: password authentication failed for user "postgres"
2026-08-14 03:14:41 UTC [4602] LOG: connection authorized: user=postgres database=tup
2026-08-14 03:19:55 UTC [4602] LOG: statement: DROP TABLE operaciones;
```

5.1 Reconstrui la cadena de hechos

5 puntos

En cinco lineas como maximo, contame que paso, en orden, desde que el grupo levanto el proyecto hasta el DROP TABLE. Sostene cada afirmacion en una linea concreta de la evidencia.

CLAVE DE CORRECCION

El grupo publico la base con -p 5432:5432 (visible en docker ps). El escaneo automatico de internet la encontro -el capitulo dice que ocurre en horas, 2.9.1-. A las 03:12 un atacante probó contraseñas contra el usuario **postgres** (tres FATAL consecutivos, dos segundos entre uno y otro: es automatizado). A las 03:14 acerto y la conexión quedó autorizada. A las 03:19 borro la tabla. **El SSH nunca participo:** grep -c "Failed password" en auth.log da 0 justamente porque el SSH estaba bien cerrado. Entraron por la puerta que el grupo abrió sin darse cuenta.

5.2 Las dos contradicciones aparentes

4 puntos (2 cada una)

La evidencia parece contradecirse dos veces. Explica cada caso: (i) ufw status dice que el 5432 esta bloqueado y nmap lo ve abierto; (ii) hubo una intrusión y sin embargo auth.log no registra un solo intento fallido.

CLAVE DE CORRECCION

(i) No hay contradicción: ufw status informa **sobre si mismo** -"tengo una regla que lo cierra"-, y esa regla vive en INPUT, cadena por la que el paquete a un puerto publicado por Docker no pasa (Parte 2). nmap informa sobre el sistema. (ii) Tampoco: auth.log registra autenticación **del sistema**, sobre todo SSH. La intrusión no uso SSH, uso PostgreSQL; por eso el rastro esta en el registro de la base y no en el del sistema. **Buscar la evidencia en el log equivocado y concluir que no paso nada es un error de metodo, no de datos.**

5.3 Los principios violados

3 puntos

De los tres principios de la seccion 2.12.1, indica cuales se violaron y en que decision concreta del grupo.

CLAVE DE CORRECCION

Minimo privilegio: la base era alcanzable por todo internet cuando solo necesitaba ser alcanzable por la API. **Valores predeterminados seguros:** el grupo dejo que el comportamiento por omision de -p (publicar en todas las interfaces) decidiera la exposicion, en vez de decidirla explicitamente. **Economia del mecanismo:** habia tres puertos publicados donde alcanzaba con uno; cuanto mas grande el mecanismo, menos verificable es. Se aceptan dos de tres bien fundamentados.

5.4 Las tres correcciones, en orden

3 puntos

Escribi las tres primeras medidas que aplicarias, en el orden en que las aplicarias. Justifica el orden, no solo el contenido.

CLAVE DE CORRECCION

Primero **dejar de publicar el 5432** (y el 8000 y el 8080): se corta el acceso ya mismo, es lo unico que detiene una intrusion en curso. Segundo **rotar la contraseña de la base y auditar que mas se toco:** la credencial esta comprometida, no solo la tabla. Tercero **rehacer el despliegue sobre red interna** para que la API alcance a la base por nombre de servicio y el mundo llegue solo por el 443. El orden importa: rotar la contraseña sin cerrar el puerto es volver a empezar el mismo juego con otra clave. Nadie suma punto por proponer `ufw deny 5432`: la Parte 2 ya demostro que no hace nada.

Parte 6 - El plan de puertos de tu propio proyecto

10 puntos

Esta parte no es un ejercicio de laboratorio: es el plano de lo que vas a desplegar en las Clases 4 y 5. El dominio ya lo tenes de la Clase 1; el servidor puede estar o no estar, no importa. Lo que define ahora es que se publica y que no, **antes** de escribir un solo `docker run`.

Servicio	Subdominio que va a usar	Puerto interno	Se publica al mundo?	Como llega el trafico	Registro DNS necesario
Panel de administracion	<code>easypanel.tudominio.me</code>	3000	Si, por HTTPS	Proxy inverso » 443, certificado emitido por Let's Encrypt	Cubierto por el comodin *
Frontend	<code>tudominio.me</code> o <code>www</code>	80 del contenedor	Si, por HTTPS	Proxy inverso » 443	Registro A raiz (y <code>www</code>)
API	<code>api.tudominio.me</code>	8000	Si, por HTTPS	Proxy inverso » 443. El 8000 no se publica	Cubierto por el comodin *
Base de datos	ninguno	5432 / 27017	No, nunca	Solo red interna: la alcanza la API por nombre de servicio	Ninguno. Lo que no se publica no se nombra
(fila libre: cache, colas, panel de metricas...)					

6.1 El registro que te ahorra trabajo

5 puntos

De todos los registros DNS de tu tabla, cuantos tuviste que crear a mano en la Clase 1 y cuantos te resuelve un solo registro? Nombralo y explica que problema evita cuando en la Clase 5 agregues un servicio nuevo.

CLAVE DE CORRECCION

El **registro comodin** (*) resuelve cualquier subdominio que no este declarado explicitamente. Con el comodin y el registro de la raiz alcanza para toda la tabla. Evita tener que volver al panel de DNS -y esperar la propagacion- cada vez que se agrega un servicio: el subdominio nuevo ya resuelve el dia que lo inventas. Es la razon por la que el capitulo 2 pide el comodin como requisito del panel (2.10) y por la que el certificado del panel no se emite si el comodin no resuelve (2.14).

6.2 La linea que te vas a agradecer

5 puntos

Mira tu tabla y responde: si manana alguien de tu grupo levanta la base "un ratito con -p 5432:5432 para probar con DBEaver desde su casa", que le decís? Escribi la respuesta tecnica **y la alternativa concreta** que le ofreces, porque la necesidad es legitima.

CLAVE DE CORRECCION

La necesidad es real y la respuesta no puede ser solo "no". La alternativa es el **tunel SSH**: `ssh -L 5432:localhost:5432 root@tudominio.me` y despues apuntar DBEaver a `localhost:5432`. El trafico viaja cifrado dentro de la conexion SSH que ya esta autenticada por clave, la base sigue sin publicar un solo puerto al mundo, y el acceso dura lo que dura la sesion. Es la segunda estrategia de 2.8.3 y el ejercicio opcional 7 del capitulo. Se valora especialmente que el alumno mencione que "un ratito" no existe: el escaneo tarda minutos, no dias (2.9.1).

Cierre - Para discutir en la mesa

sin puntaje

La conclusion del capitulo es una sola frase: **el puerto mas seguro es el que nunca se publico**. No el que esta bloqueado, no el que tiene contrasena fuerte, no el que esta en un numero raro. El que no existe hacia afuera.

Discutan cinco minutos, sin escribir: en el codigo que ya escribieron en esta carrera, donde estan sus "puertos publicados de mas"? Un endpoint que devuelve mas campos de los que la pantalla usa. Un usuario de base con permisos de administrador porque era mas rapido. Un repositorio publico con un archivo `.env` adentro. Una variable de entorno que llego al bundle del frontend. En cada uno de esos casos, cual seria el equivalente a correr `nmap` desde afuera?

Lo que se entrega

Las hojas del taller, con el nombre de los tres integrantes en la primera. Si la Parte 6 les quedo a medias, entreguenla igual como esta: esa tabla es el plano del despliegue de las Clases 4 y 5, y la vamos a retomar. Lo que **no** se entrega en blanco es la justificacion: prefiero una decision discutible bien fundamentada que una correcta sin razon.